

Modelo Basado en Agentes para el Control Descentralizado de Congestión en Redes Complejas

Universidad Sergio Arboleda
Sistemas Complejos – Parcial 1

3 de septiembre de 2026

Resumen

Este trabajo presenta una simulación computacional multiagente que estudia la aparición de la congestión como una propiedad macroscópica emergente en una red de comunicación de datos compuesta por 50 nodos, y evalúa la efectividad de un mecanismo de control descentralizado y auto-organizado basado en el cálculo de *Net Interaction* (Lafont, 1990) y heurísticas locales de contrapresión (*backpressure*) inspiradas en los principios de auto-organización de Gershenson (2007). El modelo, implementado en Python siguiendo principios SOLID y patrones de diseño GoF (*Strategy, State, Observer, Factory*), compara una estrategia de enrutamiento estático de caminos mínimos (Dijkstra) frente a una estrategia de desvío adaptativo por gradiente de carga. Se ejecutó una matriz factorial de 60 simulaciones (2 condiciones $\times$ 3 escenarios de carga $\times$ 10 semillas deterministas) sobre una topología de mundo pequeño (Watts–Strogatz, $N = 50$, $k = 4$, $p = 0,10$). Los resultados muestran que, bajo carga alta, el control distribuido reduce el descarte de paquetes en un 95,1 % y mejora la tasa de entrega en 11,43 puntos porcentuales, a costa de un incremento moderado en la latencia media, evidenciando el principio de que “ir más lento localmente permite ir más rápido globalmente”.

Índice

1. Introduucción	3
2. Marco Conceptual	3
2.1. Sistemas complejos y emergencia	3
2.2. Net Interaction (Lafont, 1990)	3
2.3. Auto-organización y contrapresión (Gershenson, 2007)	3
3. Metodología	4
3.1. Arquitectura de software	4
3.2. Definición formal del modelo	4
3.2.1. Topología	4
3.2.2. Agentes: RouterAgent	5
3.2.3. Paquetes	5
3.2.4. Ciclo de simulación (tick)	5
3.3. Estrategias de enrutamiento	5
3.3.1. Línea base: <code>BaselineShortestPathStrategy</code>	5
3.3.2. Control distribuido: <code>DistributedBackpressureStrategy</code>	6
3.4. Hiperparámetros del sistema	6
3.5. Diseño experimental	6
3.6. Métricas evaluadas	7

4. Implementación	7
4.1. Modelo central (<code>NetworkCongestionModel</code>)	7
4.2. Función de costo de la estrategia de control	8
4.3. Herramientas y tecnologías	8
5. Resultados	8
5.1. Indicadores comparativos de rendimiento	9
5.2. Dinámica temporal de la congestión (carga alta)	9
5.3. Topología espacial y estados de congestión	10
6. Discusión	11
6.1. Reducción del descarte de paquetes	11
6.2. Mejora de la tasa de entrega	11
6.3. El trade-off de la latencia: “ <i>when slow is faster</i> ”	11
6.4. Régimen de baja y media carga	12
7. Conclusiones	12
Referencias	12

1 Introducción

Las redes de comunicación de datos son sistemas complejos en los que el comportamiento macroscópico – la congestión, el colapso del tráfico, la formación de cuellos de botella – emerge de la interacción local de un gran número de componentes autónomos (enrutadores) que solo poseen información parcial de su vecindario inmediato. Este proyecto aborda el problema desde la perspectiva de los **Modelos Basados en Agentes (ABM)**, representando cada nodo de la red como un agente enrutador autónomo capaz de percibir el estado de sus vecinos directos y tomar decisiones de reenvío de paquetes en tiempo real.

El objetivo central es doble:

- (I) Caracterizar la **congestión como propiedad emergente** de un sistema de enrutamiento estático (ausencia de control), en el cual los agentes ignoran por completo el estado de saturación de sus vecinos.
- (II) Diseñar y evaluar un **mecanismo de control descentralizado** basado en información puramente local (horizonte de un salto), que permita a la red auto-organizarse y mitigar la congestión sin necesidad de un controlador central.

El fundamento teórico del mecanismo de control se apoya en dos ideas clave:

- El concepto de **Net Interaction** (Lafont, 1990), que motiva el cálculo de un costo efectivo de enrutamiento como la interacción neta entre el progreso topológico hacia el destino y la penalización por congestión local.
- El principio de auto-organización de **Gershenson (2007)**: “*when slow is faster*” – desviar tráfico por rutas ligeramente más largas pero libres de congestión resulta, en el agregado, más eficiente que insistir en el camino más corto y provocar el colapso por desbordamiento de colas.

2 Marco Conceptual

2.1 Sistemas complejos y emergencia

Una red de N enrutadores conectados según una topología de *mundo pequeño* constituye un sistema complejo adaptativo: no existe un agente con visión global de la red, y sin embargo el fenómeno de congestión – una transición de fase de un régimen fluido a un régimen saturado – emerge de la acumulación de decisiones locales simples. Este trabajo busca reproducir experimentalmente dicha transición y contrastarla con un régimen controlado en el que la retroalimentación local (backpressure) actúa como mecanismo de auto-regulación.

2.2 Net Interaction (Lafont, 1990)

La noción de *Net Interaction* formaliza el balance entre el progreso “productivo” de un paquete hacia su destino y el “costo” que introduce al sistema al competir por recursos compartidos (colas, enlaces). En el modelo, este balance se traduce en una función de costo efectivo que combina la distancia topológica remanente con penalizaciones cuadráticas por ocupación de cola.

2.3 Auto-organización y contrapresión (Gershenson, 2007)

La estrategia de control implementada no depende de coordinación global ni de un plan de enrutamiento centralizado: cada agente decide su siguiente salto exclusivamente en función del estado reportado por sus vecinos directos (ocupación de cola y estado FSM). Esta contrapresión

(*backpressure*) permite que la señal de congestión se propague orgánicamente y que el sistema se auto-organice para evitar el colapso, en línea con el principio de que la lentitud local controlada produce fluidez global.

3 Metodología

3.1 Arquitectura de software

El código fuente se organiza siguiendo principios SOLID y los siguientes patrones de diseño GoF:

- **Strategy** (`src/strategies.py`): permite intercambiar en tiempo de ejecución la política de enrutamiento entre `BaselineShortestPathStrategy` (Dijkstra estático) y `DistributedBackpressureStrategy` (desvío adaptativo por gradiente de carga).
- **State** (`src/model.py`): máquina de estados finitos que modela el nivel de saturación de cada enrutador: `NORMAL` (< 50 % de ocupación) → `ALERT` (50 %–80 %) → `CONGESTED` (≥ 80 %).
- **Observer** (`src/metrics.py`): recolector desacoplado de telemetría que captura series temporales paso a paso y computa métricas estadísticas con intervalos de confianza al 95 %.
- **Factory** (`src/topology.py`): constructor determinista y conexo de redes complejas de mundo pequeño (Watts–Strogatz).

La estructura del repositorio es la siguiente:

Listing 1: Estructura del repositorio

```

1 Parcial-1-SistemasComplejos/
2 |-- main.py                # CLI unificado
3 |-- requirements.txt
4 |-- src/
5 |   |-- config.py          # Hiperparametros y semillas
6 |   |-- model.py           # RouterAgent, Packet, NetworkCongestionModel
7 |   |-- strategies.py      # Patron Strategy (políticas de enrutamiento)
8 |   |-- topology.py        # Patron Factory (red Watts-Strogatz)
9 |   |-- metrics.py         # Patron Observer (telemetría)
10 |   |-- experiments.py     # Batch runner (60 corridas, IC 95%)
11 |   |-- plotting.py        # Figuras científicas (300 DPI)
12 |   |-- visualizer.py      # Animación interactiva en tiempo real
13 |-- data/                 # Datasets generados (CSV)
14 '-- figures/              # Figuras del informe

```

3.2 Definición formal del modelo

3.2.1. Topología

La red se genera mediante el modelo de mundo pequeño de Watts–Strogatz, garantizando conectividad total:

$$G = WS(N = 50, k = 4, p = 0,10)$$

donde N es el número de nodos, k el grado inicial en anillo regular y p la probabilidad de reconexión. Si el grafo resultante no es conexo tras 50 intentos con semillas incrementales, se conectan manualmente las componentes aisladas.

3.2.2. Agentes: RouterAgent

Cada nodo i es un agente enrutador con:

- Una cola FIFO de paquetes con capacidad máxima $C_q = 20$.
- Una capacidad de enlace (paquetes procesados por tick) $C_\ell = 2$.
- Una razón de ocupación $\rho_i = \frac{|\text{cola}_i|}{C_q}$.
- Un estado finito determinado por ρ_i :

$$\text{Estado}_i = \begin{cases} \text{NORMAL} & \rho_i < 0,50 \\ \text{ALERT} & 0,50 \leq \rho_i < 0,80 \\ \text{CONGESTED} & \rho_i \geq 0,80 \end{cases}$$

3.2.3. Paquetes

Cada paquete es una entidad discreta ($\text{id}, \text{src}, \text{dst}, t_{\text{gen}}, \text{hops}, \text{prev_hop}$) inyectada estocásticamente en la red.

3.2.4. Ciclo de simulación (tick)

En cada paso temporal t se ejecutan, en orden, las siguientes fases:

1. **Inyección de tráfico:** cada agente genera un paquete con probabilidad λ (tasa de inyección) hacia un destino aleatorio distinto de sí mismo. Si el agente está bajo régimen de control y activa el freno de inyección (*throttling*), λ se reduce a la mitad.
2. **Vaciado de búferes entrantes:** los paquetes recibidos en el tick anterior se integran a la cola principal, sujetos a la capacidad C_q .
3. **Sensado de ocupación:** cada agente actualiza su estado FSM según ρ_i .
4. **Procesamiento y reenvío concurrente:** en orden aleatorio, cada agente extrae hasta C_ℓ paquetes de su cola y delega en la *estrategia* activa la selección del siguiente salto.
5. **Actualización de estado:** se recalculan los estados FSM tras el reenvío.
6. **Registro de métricas:** el observador captura el *snapshot* del sistema.

3.3 Estrategias de enrutamiento

3.3.1. Línea base: BaselineShortestPathStrategy

Selecciona, entre los vecinos directos, aquel(los) que minimiza(n) la distancia topológica al destino según caminos mínimos precomputados (Dijkstra / BFS sobre grafo no ponderado). Ignora por completo la ocupación de colas y el estado de congestión de los vecinos.

3.3.2. Control distribuido: DistributedBackpressureStrategy

Calcula, para cada vecino v candidato, un **costo efectivo** que combina progreso topológico y penalización por congestión local (información de horizonte de un salto):

$$\text{Costo}(u \rightarrow v \mid \text{dst}) = d(v, \text{dst}) + \alpha \rho_v^2 + \beta \mathbb{I}[\text{Estado}_v = \text{CONGESTED}] + \gamma_{\text{alert}} \mathbb{I}[\text{Estado}_v = \text{ALERT}] + P_{\text{prev}} + P_{\text{prog}}$$

donde:

- $d(v, \text{dst})$ es la distancia topológica (número de saltos) del vecino v al destino.
- $\alpha = 2,0$ pondera cuadráticamente la ocupación de cola del vecino (ρ_v^2).
- $\beta = 4,0$ es la penalización fija si el vecino está en estado **CONGESTED**; se aplica $0,3\beta$ si está en **ALERT**.
- $P_{\text{prev}} = 8,0$ es una penalización anti-rebote (*anti ping-pong*) que evita reenviar el paquete al nodo del que proviene.
- $P_{\text{prog}} = 1,5$ penaliza a los vecinos que no reducen la distancia al destino respecto al nodo actual.

El siguiente salto se elige como $\arg \min_v \text{Costo}(u \rightarrow v \mid \text{dst})$, rompiendo empates aleatoriamente.

Adicionalmente, esta estrategia implementa **frenado en el origen** (*throttling*): un agente reduce a la mitad su tasa de inyección de tráfico si su propia ocupación supera el umbral de alerta, o si al menos el 50 % de sus vecinos directos está en estado **CONGESTED**.

3.4 Hiperparámetros del sistema

Tabla 1: Parámetros de configuración de la simulación

Parámetro	Valor
Número de nodos (N)	50
Grado inicial Watts–Strogatz (k)	4
Probabilidad de reconexión (p)	0.10
Capacidad de cola (C_q)	20 paquetes
Capacidad de enlace (C_ℓ)	2 paquetes / tick
Umbral de alerta	50 % de ocupación
Umbral crítico (congestión)	80 % de ocupación
Penalización cuadrática de cola (α)	2.0
Penalización por congestión (β)	4.0
Penalización anti-rebote	8.0
Razón de frenado por vecindario	50 %
Pasos de simulación por corrida	300 ticks
Réplicas por combinación (M)	10 semillas

3.5 Diseño experimental

Se ejecutó una matriz factorial completa:

2 condiciones (Sin Control / Con Control) $\times$ 3 cargas (Baja / Media / Alta) $\times$ 10 semillas = 60 simulaciones

Tabla 2: Escenarios de inyección de tráfico

Escenario	Tasa de inyección λ
Baja	0.04
Media	0.12
Alta	0.28

Las semillas deterministas utilizadas fueron: {42, 101, 202, 303, 404, 505, 606, 707, 808, 909}, garantizando reproducibilidad total de los resultados. Para cada combinación (condición, carga) se calculó la media, la desviación estándar y el intervalo de confianza al 95 % mediante la distribución t de Student:

$$IC_{95\%} = t_{0,975, n-1} \cdot \frac{s}{\sqrt{n}}$$

3.6 Métricas evaluadas

- **PDR** (*Packet Delivery Ratio*): porcentaje de paquetes generados que llegan efectivamente a su destino.
- **Throughput**: paquetes entregados por tick de simulación.
- **Latencia media**: número de ticks transcurridos entre generación y entrega de un paquete.
- **Ocupación media de colas**: longitud promedio de las colas de búfer sobre todos los nodos.
- **PCR** (*Percentage of Congested Routers*): proporción de nodos en estado CONGESTED.
- **Tiempo de recuperación**: número de ticks que tarda la red en pasar de un régimen de congestión ($PCR \geq 30\%$) a un régimen estable ($PCR \leq 5\%$).

4 Implementación

4.1 Modelo central (NetworkCongestionModel)

La clase orquestadora inicializa la topología, instancia los agentes `RouterAgent`, precomputa las distancias de caminos mínimos entre todos los pares de nodos (`all_pairs_shortest_path_length`) y administra el ciclo de simulación paso a paso, delegando la política de enrutamiento en el objeto `strategy` inyectado (patrón `Strategy`).

Listing 2: Extracto del método `step()` del modelo

```

1 def step(self):
2     # 1. Inyeccion de trafico
3     self.inject_traffic()
4     # 2. Vaciado de buferes entrantes
5     for agent in self.agents:
6         agent.flush_incoming_buffer()
7     # 3. Sensado de ocupacion (FSM)
8     for agent in self.agents:
9         agent.update_state()
10    # 4. Procesamiento y reenvio concurrente (orden aleatorio)
11    agent_order = self.agents.copy()
12    random.shuffle(agent_order)
13    for agent in agent_order:
14        agent.process_and_forward()
15    # 5. Actualizacion de estado

```

```

16     for agent in self.agents:
17         agent.update_state()
18     # 6. Registro de metricas (patron Observer)
19     self.metrics.record_step(self)
20     self.current_step += 1

```

4.2 Función de costo de la estrategia de control

Listing 3: Núcleo de DistributedBackpressureStrategy.select_next_hop

```

1  for v in neighbors:
2      neighbor_agent = agent_map[v]
3      occupancy_ratio = len(neighbor_agent.queue) / neighbor_agent.queue_capacity
4      base_dist = distances[v].get(dst_node, float('inf'))
5
6      congestion_penalty = self.alpha * (occupancy_ratio ** 2)
7      if neighbor_agent.state == NodeState.CONGESTED:
8          congestion_penalty += self.beta
9      elif neighbor_agent.state == NodeState.ALERT:
10         congestion_penalty += (self.beta * 0.3)
11
12     prev_hop_penalty = PREV_HOP_PENALTY if packet.prev_hop == v else 0.0
13     progress_penalty = 1.5 if base_dist >= current_dist else 0.0
14
15     effective_cost = base_dist + congestion_penalty + prev_hop_penalty +
        progress_penalty

```

4.3 Herramientas y tecnologías

- **Python 3.12**, NetworkX (topología y grafos), NumPy / Pandas (cómputo numérico y datos), SciPy (inferencia estadística, distribución t), Matplotlib / Seaborn (visualización científica a 300 DPI).
- CLI unificado (`main.py`) con las banderas `--experiment`, `--plot`, `--demo` y `--all`.
- Un visualizador interactivo en tiempo real (`src/visualizer.py`) basado en animaciones de Matplotlib/NetworkX, útil para la sustentación en vivo.

5 Resultados

La Tabla 3 resume los resultados consolidados de las 60 simulaciones, agregados como media $\pm$ intervalo de confianza al 95 % sobre las 10 réplicas de cada combinación (condición, carga).

Tabla 3: Resultados experimentales consolidados ($M = 10$ réplicas, IC 95 %)

Condición	Carga (λ)	Descartados	PDR (%)	Throughput	Latencia	PCR (%)
Sin Control	Baja (0.04)	$0,0 \pm 0,0$	$98,64 \pm 0,38$	$1,96 \pm 0,05$	$3,93 \pm 0,20$	0,00
Sin Control	Media (0.12)	$1,5 \pm 3,4$	$98,57 \pm 0,20$	$5,90 \pm 0,11$	$4,51 \pm 0,38$	0,02
Sin Control	Alta (0.28)	$790,2 \pm 188,4$	$77,47 \pm 4,60$	$10,83 \pm 0,63$	$11,89 \pm 1,11$	$6,15 \pm 1,22$
Con Control	Baja (0.04)	$0,0 \pm 0,0$	$98,80 \pm 0,43$	$2,01 \pm 0,06$	$3,93 \pm 0,17$	0,00
Con Control	Media (0.12)	$0,0 \pm 0,0$	$98,51 \pm 0,29$	$5,92 \pm 0,06$	$4,38 \pm 0,26$	0,00
Con Control	Alta (0.28)	$38,9 \pm 27,0$	$88,90 \pm 3,68$	$11,20 \pm 0,92$	$21,31 \pm 5,93$	$1,86 \pm 0,96$

5.1 Indicadores comparativos de rendimiento

La Figura 1 presenta los cuatro indicadores principales (PDR, throughput, latencia y ocupación de colas) para ambas condiciones a través de los tres escenarios de carga, con barras de error correspondientes al IC 95 %.

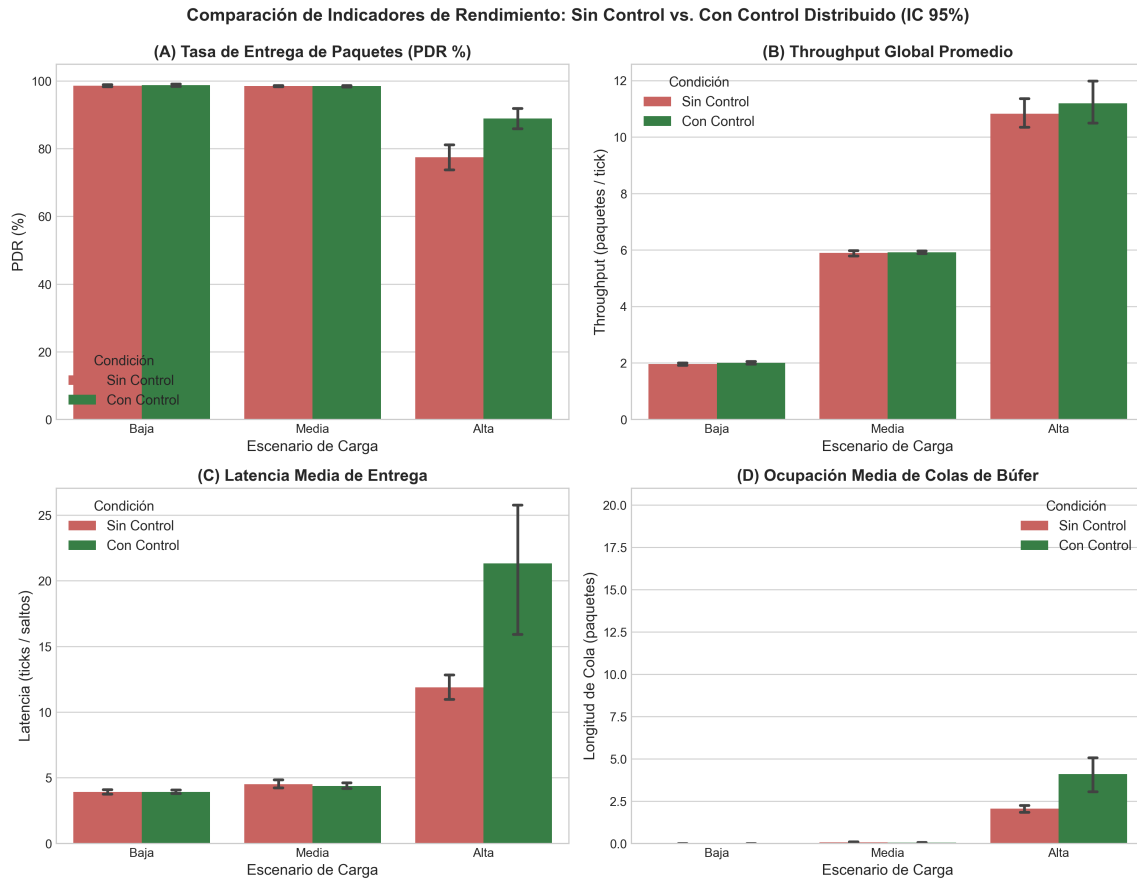


Figura 1: Comparación de indicadores de rendimiento entre la condición Sin Control y Con Control, para los tres escenarios de carga. (A) Tasa de entrega de paquetes (PDR). (B) Throughput global promedio. (C) Latencia media de entrega. (D) Ocupación media de colas de búfer.

En régimen de carga baja y media, ambas estrategias son prácticamente indistinguibles: el sistema opera muy por debajo de su capacidad y no existe presión suficiente para diferenciar el comportamiento de las políticas de enrutamiento. La diferencia sustancial emerge exclusivamente en el régimen de **carga alta**, donde el sistema Sin Control se aproxima a la saturación de sus enlaces y colas.

5.2 Dinámica temporal de la congestión (carga alta)

La Figura 2 muestra la evolución temporal de la longitud media de cola y de la proporción de nodos congestionados (PCR) durante los 300 ticks de simulación, bajo el escenario de carga alta.

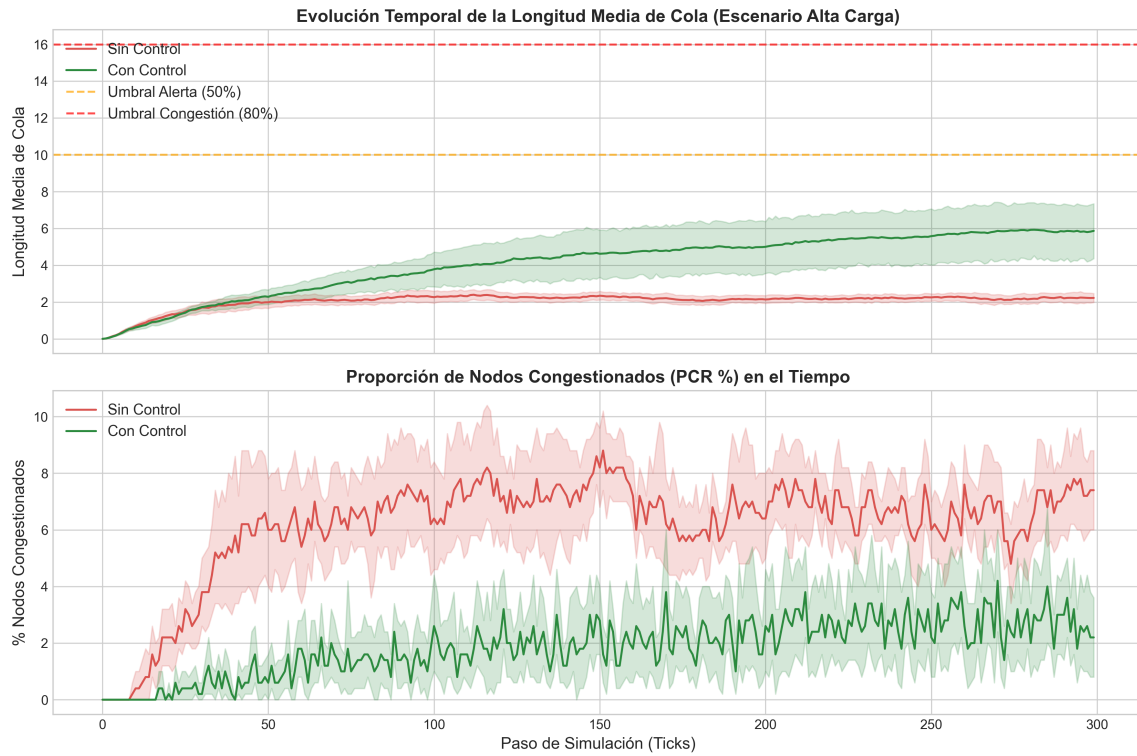


Figura 2: Dinámica temporal de la congestión bajo carga alta ($\lambda = 0,28$). Panel superior: longitud media de cola con umbrales de alerta (50 %) y congestión (80 %). Panel inferior: proporción de nodos congestionados (PCR) en el tiempo.

Se observa que, sin control, la red exhibe una **transición abrupta hacia el colapso**: la longitud media de cola crece de forma prácticamente monótona hasta estabilizarse cerca del límite de capacidad, arrastrando consigo un aumento sostenido del PCR. En contraste, bajo control distribuido la red logra **auto-regularse**: los picos de ocupación son absorbidos mediante desvíos adaptativos y frenado de inyección en el origen, manteniendo la mayoría de los nodos en régimen **NORMAL** o **ALERT** en lugar de **CONGESTED**.

5.3 Topología espacial y estados de congestión

La Figura 3 presenta una instantánea (*snapshot*) de la red completa en el tick 150 bajo carga alta, coloreando cada nodo según su estado FSM (verde = **NORMAL**, naranja = **ALERT**, rojo = **CONGESTED**) y escalando su tamaño según la longitud de su cola.

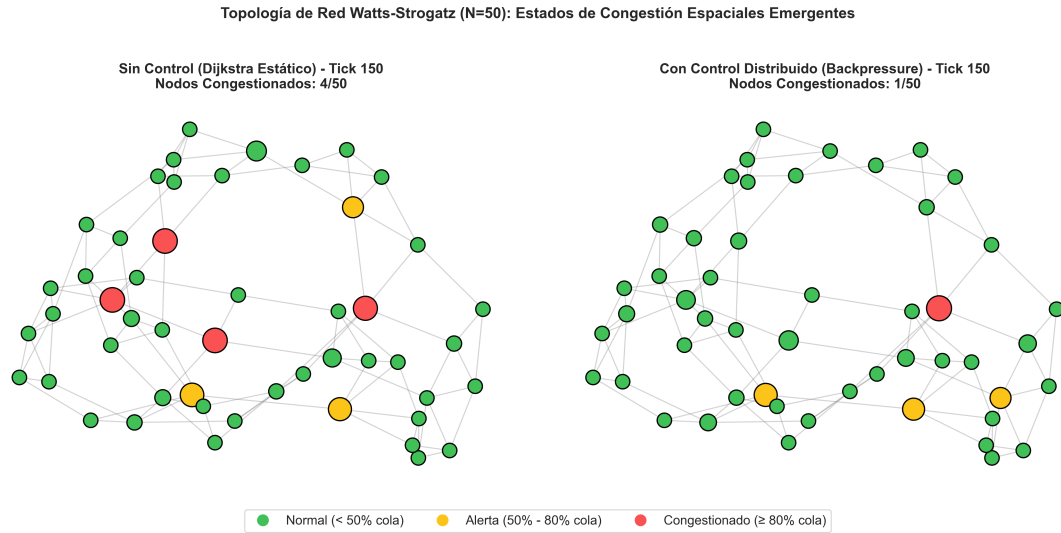


Figura 3: Snapshot espacial de la topología Watts–Strogatz ($N = 50$) en el tick 150, bajo carga alta. Izquierda: Sin Control (Dijkstra estático). Derecha: Con Control Distribuido (backpressure).

Esta visualización evidencia el carácter **emergente y espacialmente heterogéneo** de la congestión: los nodos con mayor centralidad de intermediación (*betweenness centrality*) actúan como cuellos de botella naturales de la topología, concentrando la mayor parte de los estados críticos en el escenario sin control.

6 Discusión

6.1 Reducción del descarte de paquetes

En el régimen saturado (carga alta), el control distribuido reduce el número de paquetes descartados por desbordamiento de cola de $790,2 \pm 188,4$ a solo $38,9 \pm 27,0$ paquetes, lo que representa una **reducción del 95.1 %** en la pérdida de información.

6.2 Mejora de la tasa de entrega

La tasa de entrega de paquetes (PDR) asciende de 77,47 % a 88,90 %, una mejora de **+11,43** puntos porcentuales, confirmando que el mecanismo de contrapresión logra sostener el rendimiento global del sistema incluso cuando la demanda de tráfico se acerca a la capacidad física de la red.

6.3 El trade-off de la latencia: “*when slow is faster*”

El incremento en la latencia media observado bajo control ($11,89 \rightarrow 21,31$ ticks) no debe interpretarse como una degradación del sistema, sino como el **coste temporal óptimo** que el mecanismo de contrapresión impone deliberadamente: al desviar paquetes por rutas secundarias más largas pero libres de congestión, se evita la destrucción de información por descarte masivo. Este resultado reproduce empíricamente el principio de auto-organización descrito por Gershenson (2007): en sistemas de tráfico saturados, permitir que los componentes individuales se muevan más lento (mayor latencia) produce, en el agregado, un sistema más rápido y estable (mayor throughput efectivo y menor pérdida).

6.4 Régimen de baja y media carga

La ausencia de diferencias significativas en los escenarios de baja y media carga confirma que el mecanismo de control no introduce sobrecostos apreciables cuando la red opera lejos de su punto de saturación: el sistema de contrapresión es *transparente* en condiciones normales y solo se activa de forma efectiva cuando la presión sobre las colas lo amerita.

7 Conclusiones

1. La congestión en la red simulada emerge como una **propiedad macroscópica** del sistema, resultado de decisiones de enrutamiento puramente locales y estáticas que ignoran la retroalimentación de estado de los vecinos.
2. Un mecanismo de **control descentralizado** basado exclusivamente en información de horizonte de un salto (estado y ocupación de los vecinos directos) es suficiente para lograr una mejora sustancial en la resiliencia de la red frente a la saturación, sin requerir coordinación global ni un controlador central.
3. El diseño del sistema mediante los patrones **Strategy**, **State**, **Observer** y **Factory** permitió aislar de forma limpia la política de enrutamiento, la lógica de estados de congestión, la recolección de métricas y la generación de la topología, facilitando la comparación experimental rigurosa entre condiciones.
4. Los resultados validan empíricamente, en el contexto de redes de datos, los principios de auto-organización de Gershenson (2007) y el balance de interacción neta de Lafont (1990): la lentitud local controlada, lejos de ser una falla, constituye un mecanismo eficiente de regulación global en sistemas complejos saturados.

Referencias

- Lafont, Y. (1990). *Interaction Nets*. Proceedings of the 17th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL).
- Gershenson, C. (2007). *Design and Control of Self-organizing Systems*. CopIt ArXives.
- Watts, D. J., & Strogatz, S. H. (1998). *Collective dynamics of ‘small-world’ networks*. Nature, 393(6684), 440–442.
- Hagberg, A. A., Schult, D. A., & Swart, P. J. (2008). *Exploring network structure, dynamics, and function using NetworkX*. Proceedings of the 7th Python in Science Conference.